

MobileNetV3 Training

A beginner-friendly guide to the project files, preparing a crop-image dataset, training the model, evaluating it, and exporting it for Android.

What this folder is

A complete recipe for teaching a compact image-recognition model to classify 15 maize, cassava, and rice conditions. It contains training code, but it does not yet contain the image dataset or a trained model.

Prepared from the current **mobilenetv3-training** project configuration.

0. How to use this guide

Read Sections 1-3 to understand the project. Use Sections 4-8 as the practical checklist when the image dataset is ready.

Section	What you will learn
1. The big picture	What model training means and how information moves through the pipeline
2. Model contract	What enters the model and what comes back
3. Project files	What every file and helper module is responsible for
4. Dataset plan	How to collect useful, ethical, realistic crop photos
5. Dataset setup	Exactly where images and plant/session groups belong
6. Environment setup	How to install Python dependencies and validate the project
7. Train and evaluate	Commands to prepare, train, test, and inspect results
8. Export and integrate	How to make the Android-ready TFLite model
9. Troubleshooting	Common beginner problems and what they mean
10. Launch checklist	A final checklist before trusting the model in the field

Fastest safe path

Collect and group the images first. Then run: validate setup, prepare dataset, train, evaluate, export. Do not judge model quality from training accuracy alone; the held-out test set is the real exam.

1. The big picture

Training is supervised learning: each photo is paired with the correct answer.

Imagine teaching a student with flash cards. You show a leaf photo, tell the student its diagnosis, and repeat this with many different examples. MobileNetV3 changes millions of small internal numbers until its guesses improve.

1	Labeled images	Photos are stored in folders named with the correct diagnosis.
2	Prepare	The code checks, catalogs, and splits images without mixing the same plant across sets.
3	Train	The model studies training images and adjusts itself.
4	Validate	A separate set helps decide when training should stop.
5	Test	Held-out images measure how well the model handles unseen examples.
6	Export	The best model becomes a compact .tflite file for Android.

Transfer learning saves time and data. The project starts from MobileNetV3Small, which has already learned general visual features from ImageNet. It then learns crop-specific patterns instead of starting from a blank slate.

Two-stage training makes that adaptation safer. First, the pretrained MobileNetV3 backbone is frozen and only a new 15-class classification head learns. Second, the last 30 backbone layers are carefully fine-tuned using a much smaller learning rate.

Important limitation

The model predicts a condition class, not disease severity, treatment, or certainty that the photo is safe to diagnose. Those are separate product and validation decisions.

Training, validation, and test sets

Set	Configured share	Beginner analogy	Purpose
Training	70%	Study material	Used to change the model's internal weights
Validation	15%	Practice exam	Used during training to choose the best checkpoint
Test	15%	Final exam	Used once training is complete to estimate performance

2. What goes into the model - and what comes back

Each input is one RGB color image resized to **224 x 224 pixels**. Pixel values remain in the 0-255 range because MobileNetV3's normalization is embedded inside the model.

Part	Shape or format	Meaning
Input	1 x 224 x 224 x 3	One image, 224 pixels high and wide, with red/green/blue channels
Output	1 x 15	Fifteen probability scores in the exact order defined by classes.json
Decision	Highest score	Usually displayed as the predicted crop condition

Example output (simplified):

```
maize_healthy           0.03
maize_common_rust       0.82  <- highest score
maize_northern_leaf_blight 0.08
all other classes combined 0.07
```

The 15 supported classes

Index	Crop	Diagnosis ID
0	Maize	maize_healthy
1	Maize	maize_streak_disease
2	Maize	maize_northern_leaf_blight
3	Maize	maize_common_rust
4	Maize	maize_unknown
5	Cassava	cassava_healthy
6	Cassava	cassava_brown_streak_disease
7	Cassava	cassava_bacterial_blight
8	Cassava	cassava_mosaic_disease
9	Cassava	cassava_unknown
10	Rice	rice_healthy
11	Rice	rice_bacterial_leaf_blight
12	Rice	rice_blast
13	Rice	rice_yellow_mottle_disease
14	Rice	rice_unknown

3. What is inside mobilenetv3-training

The top-level files are the controls you run. The **training/** folder is the engine room they share. The **tests/** folder checks that important assumptions stay true.

File	Plain-language job
README.md	The official setup and command instructions.
classes.json	The fixed list and order of the 15 answers the model can return.
config.json	Training settings: image size, batch size, split percentages, epochs, and learning rates.
requirements.txt	Pins TensorFlow 2.16.2 so the software environment is repeatable.
.python-version	Records Python 3.12 as the expected Python version.
groups.example.csv	Shows how to identify photos from the same plant or capture session.
validate_setup.py	Checks Python, TensorFlow, class labels, and model shapes before training.
prepare_dataset.py	Checks images and creates repeatable train, validation, and test lists.
train.py	Trains the new classifier head, fine-tunes MobileNetV3, and keeps the better checkpoint.
evaluate.py	Measures held-out performance and records per-class metrics and a confusion matrix.
export_tflite.py	Creates verified float32 and float16 .tflite files for Android.

Inside training/

File	Plain-language job
contract.py	Protects the 15-class agreement between training and the mobile app.
manifest.py	Finds images, detects duplicates, checks groups, validates files, and performs the split.
data.py	Loads and resizes images; adds flips, rotation, zoom, and contrast changes during training.
model.py	Builds MobileNetV3Small, adds the 15-class head, and controls fine-tuning.
__init__.py	Marks the directory as an importable Python package.

Inside tests/

- **test_model.py:** verifies that the model accepts 224 x 224 RGB images, produces 15 outputs, and begins with a frozen backbone.
- **test_training_contract.py:** verifies class order, mobile-label agreement, and group-safe data splitting.
- **__init__.py:** marks the test directory as a Python package.

4. Plan and collect the dataset

The quality of the photos and labels matters more than simply having a sophisticated model.

1**Define the label rules**

Write a short visual definition for every class. Decide what counts as healthy, each disease, and unknown. Have a crop expert review ambiguous cases whenever possible.

2**Collect independent plants**

Photograph many different plants, farms, regions, devices, seasons, lighting conditions, backgrounds, leaf ages, camera distances, and disease stages. Ten images per class only tests whether the code runs; it is not enough for a reliable field model.

3**Avoid shortcuts**

Do not let one class come only from one farm, phone, background, or website. Otherwise the model may learn the background or camera instead of the disease.

4**Record provenance**

For every photo, keep its diagnosis source, plant or session ID, location or region where appropriate, capture date, crop, and usage permission. Remove personal or sensitive information.

5**Review labels**

Use expert confirmation where feasible. Flag uncertain images instead of forcing them into a disease class. Duplicate and near-duplicate photos should not inflate the dataset.

6**Assign plant/session groups**

All photos of the same physical plant, video burst, or capture session must share one group ID. This keeps them in one split and makes the final exam honest.

What belongs in an unknown class?

Each unknown folder should contain the named crop when it does not reliably match the supported healthy or disease classes. Useful examples include other diseases, nutrient deficiencies, pest damage, mixed symptoms, and visually ambiguous cases. Do not use random non-crop images as the only unknown examples.

Data leakage

If nearly identical images of the same plant appear in both training and testing, the model can memorize that plant. The reported score may look excellent while real-world performance remains poor.

Practical collection target

There is no universal magic number. Start with a pilot dataset large enough to expose labeling and pipeline problems, then expand based on per-class test results. For field use, plan for hundreds to thousands of independently captured images per class when possible, with balanced representation across real deployment conditions.

5. Put the dataset into the project

From the **mobilenetv3-training** directory, create **datasets/raw/** and one folder for each diagnosis ID. Use JPEG or PNG images.

```
datasets/raw/
  maize_healthy/
  maize_streak_disease/
  maize_northern_leaf_blight/
  maize_common_rust/
  maize_unknown/
  cassava_healthy/
  cassava_brown_streak_disease/
  cassava_bacterial_blight/
  cassava_mosaic_disease/
  cassava_unknown/
  rice_healthy/
  rice_bacterial_leaf_blight/
  rice_blast/
  rice_yellow_mottle_disease/
  rice_unknown/
```

A file's folder is its answer

If **leaf-0042.jpg** truly shows maize common rust, place it in **datasets/raw/maize_common_rust/**. Folder names must match **classes.json** exactly.

Create datasets/groups.csv

Copy the idea from **groups.example.csv**. Give every image a stable group ID representing the physical plant or capture session. Multiple photos of the same plant use the same ID.

```
relative_path,group_id
datasets/raw/cassava_healthy/image-001.jpg,farm-12-plant-7-session-1
datasets/raw/cassava_healthy/image-002.jpg,farm-12-plant-7-session-1
datasets/raw/rice_blast/image-310.jpg,farm-4-plant-19-session-2
```

Required for trustworthy evaluation

The script can run without groups.csv, but it will treat every image as independent. That fallback is acceptable only for pipeline testing, not for a final performance claim.

Before moving on

- Every image has a defensible label and usage permission.
- All 15 folders exist and contain images.
- Every class has at least three independent groups so it can be represented across the three splits.
- Repeated shots of one plant share one group ID.
- The unknown folders contain meaningful in-crop alternatives and ambiguity.
- Classes are reasonably balanced; severe imbalances are understood and documented.

6. Set up the training environment

Open Terminal and move into the training folder. The commands below create an isolated Python environment so this project's exact TensorFlow version does not interfere with other projects.

```
cd /Users/zion/Projects/UCLA/Hacknation/mobilenetv3-training
```

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

What these commands mean

Command	Plain-language meaning
<code>cd ...</code>	Move Terminal into the training project directory.
<code>python3.12 -m venv .venv</code>	Create a private Python workspace named <code>.venv</code> .
<code>source .venv/bin/activate</code>	Turn on that private workspace for this Terminal window.
<code>pip install --upgrade pip</code>	Update Python's package installer inside the private environment.
<code>pip install -r requirements.txt</code>	Install the project's pinned TensorFlow dependency.

Validate before training

```
python validate_setup.py
python -m unittest discover -s tests -v
python -m compileall -q .
```

- **validate_setup.py** checks Python, TensorFlow, labels, and model input/output shapes.
- **unittest** runs the automated contract and model checks.
- **compileall** catches basic Python syntax errors.

If TensorFlow is not installed yet

You can still verify the class and mobile-label contract with: `python validate_setup.py --contract-only`

7. Prepare the data and start training

Keep the Python virtual environment active and run commands from the **mobilenetv3-training** folder.

Step A - Build the manifests

```
python prepare_dataset.py
```

This checks folder names, minimum counts, unreadable images, byte-identical duplicates, group validity, and the reproducible train/validation/test split. It writes CSV manifests and a summary under **manifests/**. Read the output carefully before training.

Step B - Start training

```
python train.py
```

The first phase trains only the new 15-class head for up to 20 epochs with a learning rate of 0.001. The second phase fine-tunes the last 30 MobileNetV3 layers for up to 15 epochs with a smaller learning rate of 0.00001.

An **epoch** is one pass through the training set. A **batch** is a smaller group of images processed together; this project uses batches of 32. Early stopping can finish before the maximum number of epochs when validation performance stops improving.

Helpful protections already included

- **Data augmentation:** creates reasonable variations using flips, small rotations, zoom, and contrast changes.
- **Class weights:** gives rarer classes more influence when the dataset is imbalanced.
- **Early stopping:** stops when validation loss no longer improves.
- **Learning-rate reduction:** takes smaller adjustment steps when progress stalls.
- **Best checkpoint selection:** compares the head-only and fine-tuned models and keeps the one with lower validation loss.

What training creates

Directory	Contents
manifests/	Train, validation, and test image lists plus dataset counts.
checkpoints/	Saved Keras models and metadata; best_model.keras is the selected model.
runs/	Training histories, logs, and later evaluation reports.

Hardware expectation

Training can run on a CPU, but it may be slow. A supported GPU can reduce training time considerably. The exact duration depends mostly on image count, computer hardware, storage speed, and whether TensorFlow can use acceleration.

8. Evaluate, export, and prepare for Android

Evaluate the hidden test set

```
python evaluate.py
```

The script writes **runs/evaluation/metrics.json** and **confusion_matrix.csv**. These results matter more than training accuracy because the test images were not used to adjust the model.

Metric	Plain-language meaning
Accuracy	The percentage of test images whose top guess was correct.
Top-3 accuracy	How often the correct answer appeared among the three highest guesses.
Precision	When the model predicts a class, how often that prediction is right.
Recall	Of all true examples of a class, how many the model successfully found.
F1 score	A combined summary of precision and recall.
Confusion matrix	A grid showing which classes are mistaken for which other classes.

Do not rely on one overall percentage. Review every class. A model can have high overall accuracy while performing poorly on a rare but important disease.

Export the Android-ready models

```
python export_tflite.py
```

The export step converts **checkpoints/best_model.keras** into two TensorFlow Lite models, verifies the expected 224 x 224 x 3 input and 15-value output, performs a test inference, and packages matching labels and preprocessing metadata.

Export	Tradeoff
float32	Larger file and full numerical precision; useful as the baseline.
float16	Smaller file and often better suited to mobile deployment; verify accuracy and speed on target phones.

The resulting files appear under **exports/**. Before replacing a model in the Android app, keep the exported model, labels, and metadata together. The label order must remain exactly synchronized with the app's label file.

Confidence is not a diagnosis guarantee

Softmax scores rank the 15 known choices, but they are not automatically calibrated probabilities of medical or agronomic correctness. Choose any low-confidence rejection threshold only after evaluation on representative field data.

9. Troubleshooting common beginner problems

What you see	Likely meaning and next check
Python 3.12 not found	Install Python 3.12 or select an existing compatible interpreter before creating .venv.
TensorFlow version mismatch	Activate .venv and reinstall from requirements.txt; the project expects TensorFlow 2.16.2.
Missing class directory	Create every folder listed in classes.json, with the spelling unchanged.
Too few images	Add valid, independently captured images. The configured mechanical minimum is 10 per class.
Fewer than 3 groups	Add independent plants or sessions. A class needs groups that can be assigned across train, validation, and test.
Unreadable image	Open or re-export the reported image; remove it if corrupted.
Duplicate warning	Check whether the same file was copied more than once. Keep only the intended example.
Validation improves, test is poor	The validation set may not represent deployment, or leakage/overfitting may be present. Audit groups and collect broader data.
One class is rarely detected	Inspect its recall, label quality, visual overlap, and dataset diversity; collect targeted examples.
Training loss becomes NaN	Look for bad data or unstable settings. The current training code stops on NaN to avoid producing a misleading model.
Android output labels are wrong	The app label order probably differs from classes.json/exported labels. Treat order as part of the model contract.

If a run is interrupted

Do not assume the newest file is the best model. Check the logs and **checkpoints/**. The normal pipeline selects **best_model.keras** based on validation loss. Restart only after understanding whether the prior process completed and whether its outputs are valid.

10. Before calling the model field-ready

Check	Ready when...
Labels	Definitions are documented and ambiguous examples were reviewed.
Permissions	Every image is legally and ethically usable for the intended product.
Independence	The same plant/session never crosses train, validation, and test sets.
Diversity	Photos reflect real devices, farms, lighting, backgrounds, stages, and regions.
Unknowns	Unknown classes contain realistic alternatives for each named crop.
Per-class results	Precision, recall, F1, and confusion patterns are acceptable for every important class.
External test	The model has been tested on newly collected data from outside the training source.
Device test	The exported TFLite model runs correctly and fast enough on representative Android phones.
Safety UX	The app communicates uncertainty, limitations, and appropriate next steps.
Monitoring	There is a plan to collect failures, review labels, retrain, and version future models.

Command sequence - quick reference

```
cd /Users/zion/Projects/UCLA/Hacknation/mobilenetv3-training
source .venv/bin/activate
```

```
python validate_setup.py
python prepare_dataset.py
python train.py
python evaluate.py
python export_tflite.py
```

Mini glossary

Term	Meaning
Model	A learned mathematical system that turns an image into scores.
Label	The correct class assigned to a training photo.
Epoch	One complete pass through the training images.
Learning rate	How large each model adjustment is during training.
Checkpoint	A saved copy of the model at a particular point in training.
Overfitting	Learning the training examples too specifically and performing poorly on new photos.
TFLite	TensorFlow Lite, a model format designed for phones and other edge devices.

Bottom line

The code pipeline is ready. The next major work is building a clean, diverse, correctly labeled, group-aware image dataset and then judging the model on genuinely unseen field photos.